

Protokoll – “TourPlaner”

Lehrveranstaltung: Software Engineering 2 (SWEN2)

Semesterprojekt: Tour Planner

Datum: 06.07.2026

Team / Mitwirkende: Abdullah Hakimi, Humza Ajaz

Repository: <https://github.com/Abdullah10-swen/tour-planner-team09>

1. Einleitung

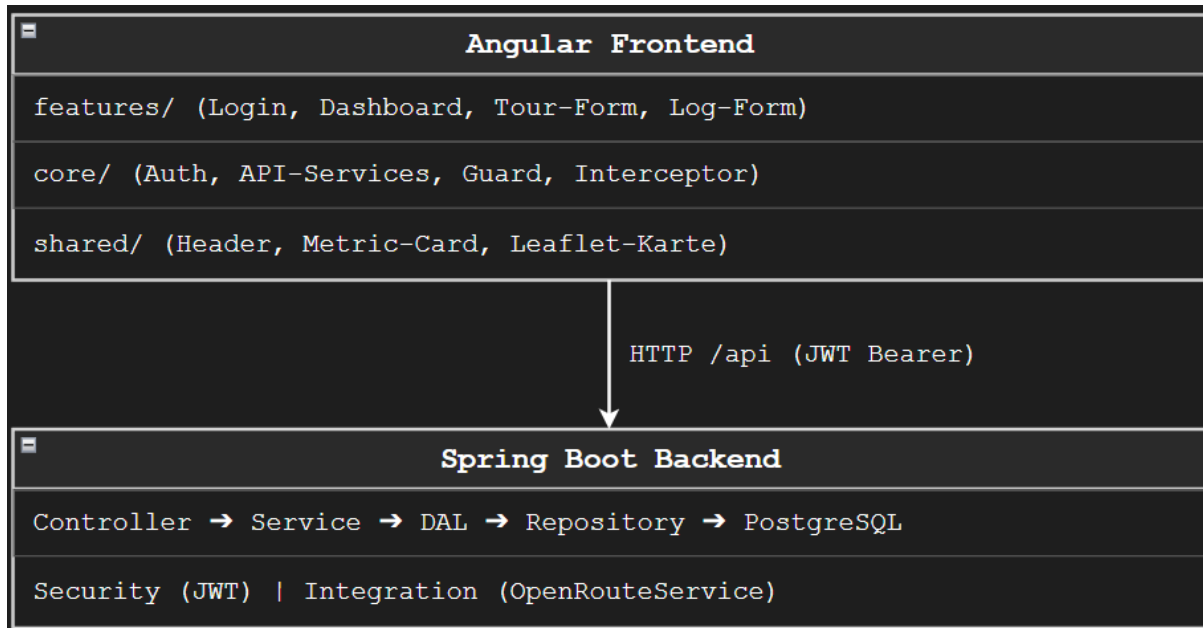
Dieses Protokoll dokumentiert die Entwicklung des Projekts „Tour Planner“, einer webbasierten Client-Server-Anwendung zur Planung und Verwaltung von personalisierten Touren und Aktivitäten. Die Applikation richtet sich an Endnutzer, die verschiedene Arten von Touren – wie Wanderungen, Fahrradtouren, Laufstrecken oder Urlaubsreisen – im Voraus planen sowie die dazugehörigen Erfahrungsberichte (Tour-Logs) und Statistiken zentral verwalten möchten.

2. App-Architektur

2.1 Gesamtüberblick

Die Anwendung ist eine **Client-Server-Architektur** mit getrenntem Frontend und Backend:

Schicht	Technologie	Aufgabe
Frontend	Angular 21 (Standalone Components)	UI, Routing, HTTP-Kommunikation, Kartenanzeige
Backend	Spring Boot 4 (Java 25)	REST-API, Business-Logik, Persistenz, Sicherheit
Datenbank	PostgreSQL (via Docker)	Persistente Speicherung von Usern, Touren, Logs
Externe API	OpenRouteService	Geocoding, Routenberechnung, GeoJSON



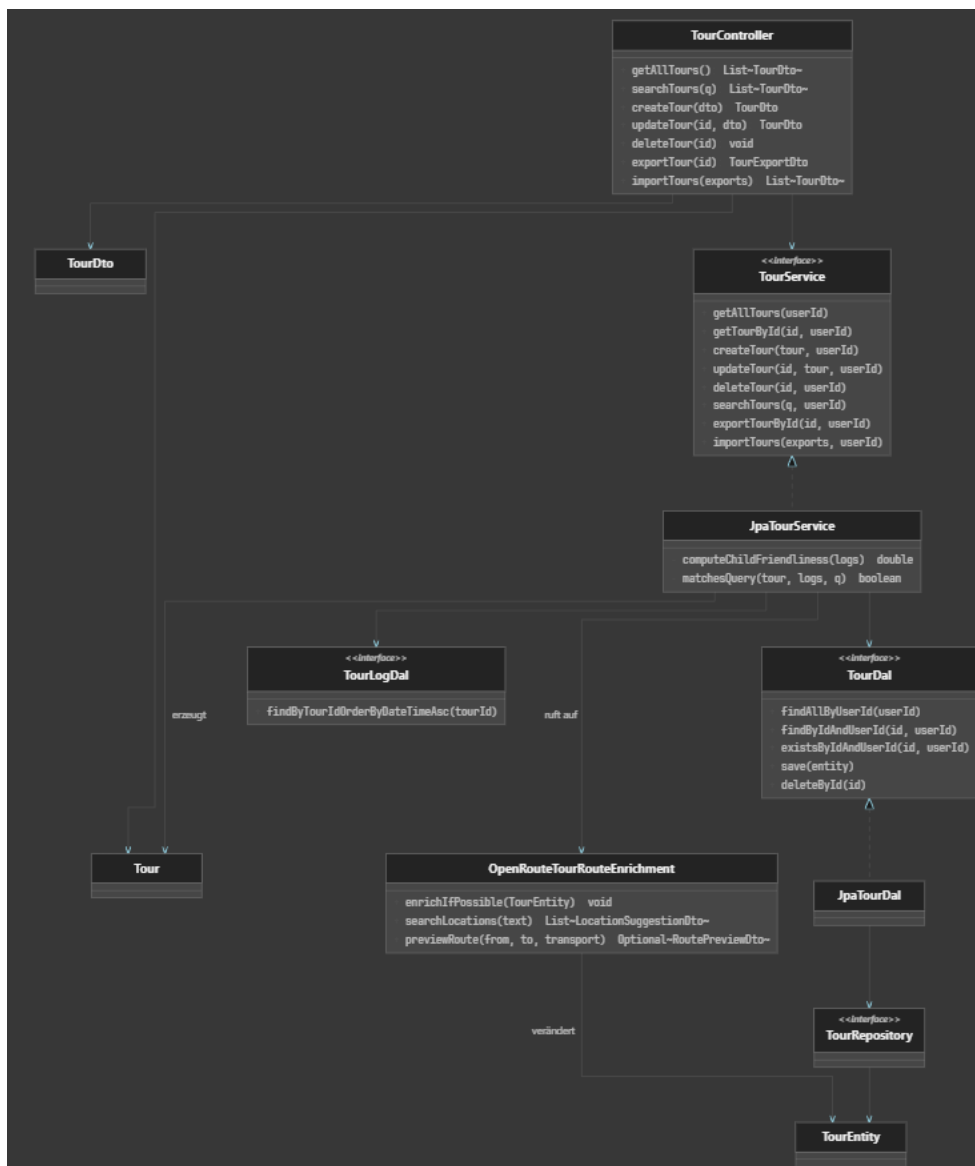
2.2 Backend-Schichten

Schicht	Paket	Inhalt / Funktion
Controller	controller/	REST-Endpunkte, DTO-Mapping, userId aus JWT
Service	service/, service/Impl/	Geschäftslogik, Transaktionen, berechnete Metriken
DAL	dal/, dal/Impl/	Abstraktion der Persistenz, Fehlerübersetzung
Repository	dal/repository/	Spring Data JPA (JpaRepository)
Entity	dal/entity/	JPA-Entitäten (TourEntity, TourLogEntity, UserEntity)
Model	model/	Domänenobjekte ohne JPA-Abhängigkeit
DTO	dto/	API-Request/Response-Objekte
Security	security/	JWT-Filter, Spring Security Config
Integration	integration/openroute/	OpenRouteService-Anbindung

2.3 Frontend-Schichten

Schicht	Ordner	Inhalt / Funktion
Features	features/	Routen-Komponenten (Login, Dashboard, Formulare)
Core	core/	Auth, API-Services, Models, Guard, Interceptor
Shared	shared/components/	Wiederverwendbare UI-Komponenten (Header, Karte, Metriken)

2.4 Klassendiagramm (Backend, vereinfacht)

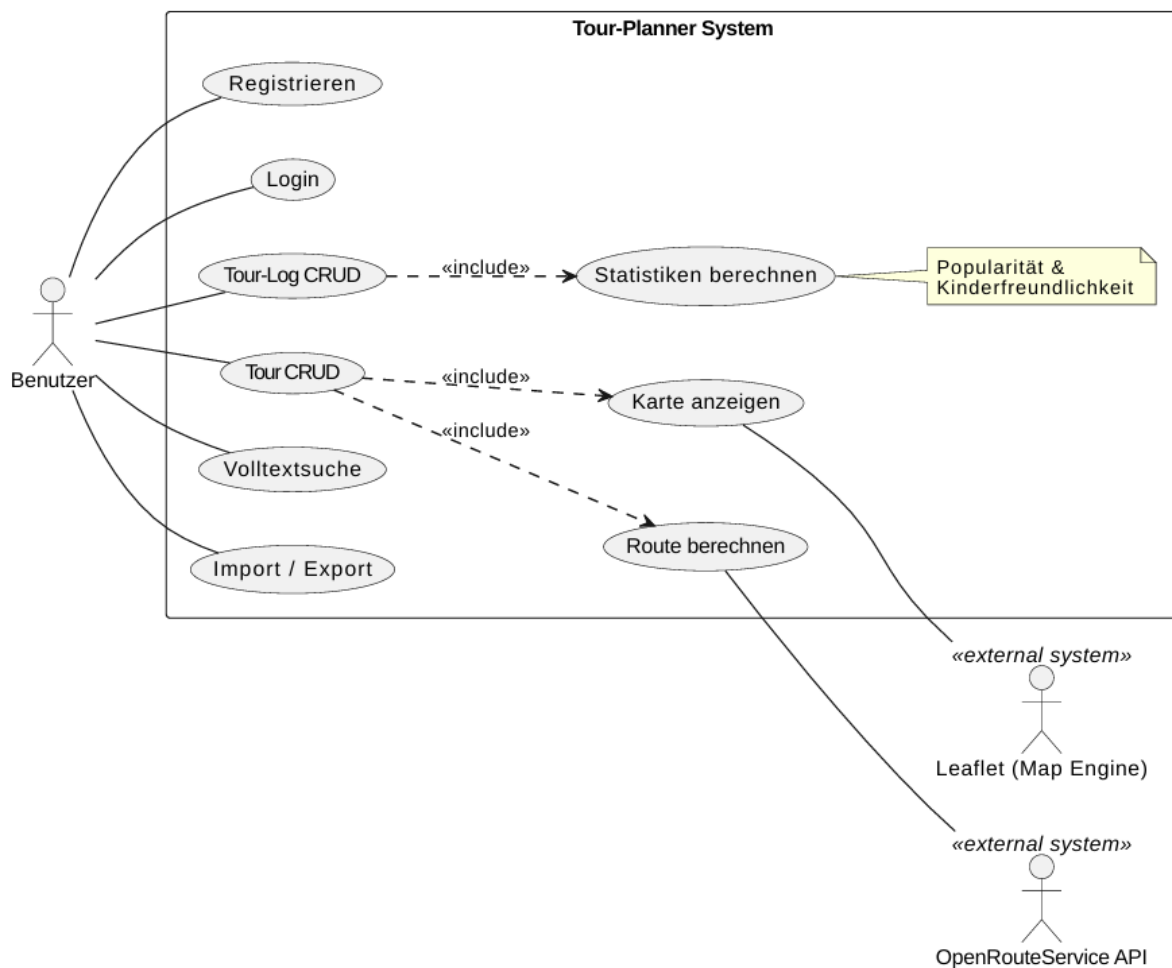


3. Use Case

3.1 Akteure

- Registrierter Benutzer – plant Touren, führt Logs, nutzt Suche und Export
- Gast (nicht eingeloggt) – nur Login/Registrierung
- OpenRouteService – externes System für Geocoding und Routing

3.2 Use-Case-Diagramm



4. UX-Beschreibung (User Experience)

4.1 Zielgruppe und Nutzungskontext

Die Anwendung richtet sich an sportbegeisterte Nutzer, die Touren planen und absolvierte Aktivitäten per Log festhalten. Die Oberfläche soll ruhig, lesbar und mobilfähig sein (responsive Layout, z. B. Sidebar untereinander auf schmalen Viewports).

4.2 Informationsarchitektur / Navigation

- Start: Dashboard (/) – Liste „Meine Touren“, Auswahl einer Tour, Detailbereich mit Kennzahlen und Log-Liste sowie „+ Neue Tour“.
- Neue Tour: /tours/new – Formular links, Karten-Platzhalter rechts; Speichern/Abbrechen in der Kopfzeile.
- Tour bearbeiten: /tours/:id/edit – gleiche Formularstruktur mit vorausgefüllten Daten.
- Neuer Log: /tours/:tourId/logs/new – Kontext der Tour (Name, Kennzahlen), Eingabe der Log-Felder.
- Log bearbeiten: /tours/:tourId/logs/:logId/edit.

4.3 Login- und Registrierungs-Flow

- Einstiegspunkt: /login – zweigeteiltes Layout mit Hero-Panel links (Branding „TourPlaner“) und Formular-Panel rechts.
- Modus-Umschalter: Zwei Tabs „Einloggen“ / „Registrieren“ oben im Formular-Panel; Umschalten setzt alle Eingabefelder und Fehlermeldungen zurück.
- Login: Felder „Benutzername“ und „Passwort“ → Klick auf „Einloggen“ sendet POST /api/auth/login; bei Erfolg wird das JWT im localStorage gespeichert und zum Dashboard (/) weitergeleitet.
- Registrierung: Zusätzliche Felder „E-Mail-Adresse“ und „Passwort (wiederholen)“; Client-seitige Validierung vor dem Absenden (alle Felder ausgefüllt, Passwörter identisch, Passwort ≥ 6 Zeichen) → POST /api/auth/register → bei Erfolg automatischer Login und Weiterleitung zum Dashboard.
- Fehlerbehandlung: Serverseitige Fehler werden in deutschen, kontextspezifischen Meldungen angezeigt (z. B. „Ungültige Zugangsdaten.“ bei 401, „Benutzername oder E-Mail bereits vergeben.“ bei 409).
- Ladezustand: Submit-Button zeigt „Bitte warten...“ und ist deaktiviert, solange die Anfrage läuft.
- Routen-Absicherung: Ein authGuard prüft vor jedem geschützten Routenwechsel, ob ein gültiges Token vorhanden ist; ohne Token erfolgt automatische Weiterleitung zu /login. Ein authInterceptor hängt das JWT (Authorization: Bearer ...) an jeden ausgehenden API-Request an.
- Logout: Über das Benutzermenü im Header (AppHeaderComponent) – löscht Token aus dem localStorage und leitet zurück zu /login.

4.4 Visuelles Konzept

- Farben: helles Hintergrund-Beige/Creme, Waldgrün für Primäraktionen, dezente Grautöne für Flächen und Rahmen.
- Typo: Serif für App-Namen/Überschriften, Sans-serif für UI-Texte (Orientierung an Mockup).
- Komponenten: stark abgerundete Buttons und Eingabefelder; wiederverwendbare Elemente z. B. Header (Suche, „Neue Tour“) und Kennzahlen-Karten (MetricCard).

4.5 Feedback & Validierung

Pflichtfelder in Formularen mit Reactive Forms; bei ungültiger Eingabe kein Absturz, sondern Hinweise bzw. deaktiviertes Speichern wo sinnvoll. Fehlertext, wenn das Backend nicht erreichbar ist.

5. Wireframes

Die folgenden Abbildungen 1–5 im Abschnitt 5.1 zeigen die geplanten Oberflächen als High-Fidelity-Mockups – das entspricht den geforderten Wireframes bzw. dem UX-Design vor der Implementierung. [Figma-Link](#)

Desweiteren ergänzen wir einen illustrativen Vergleich zwischen Design (Mockup) und der laufenden Angular-Oberfläche.

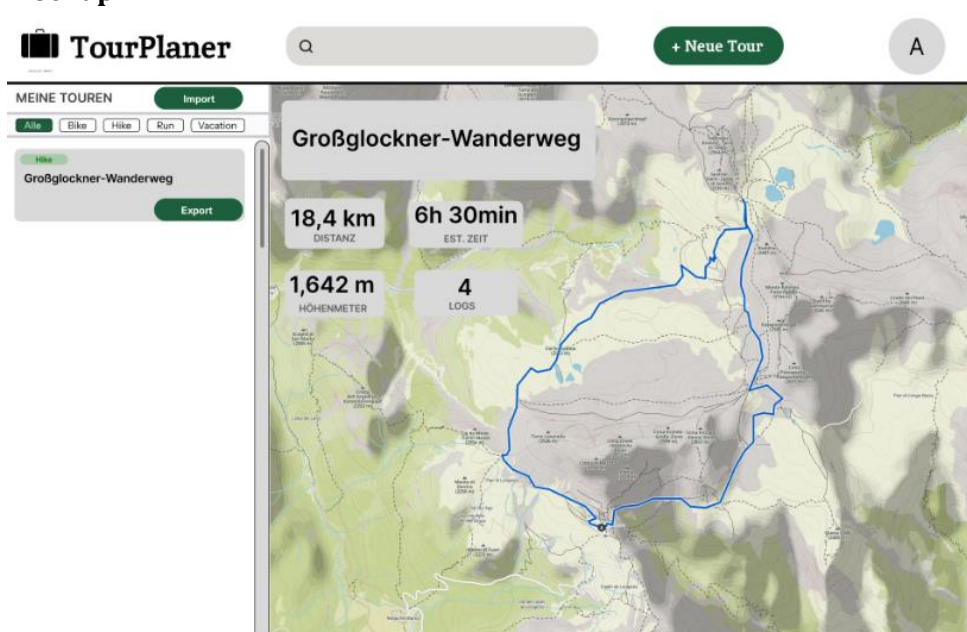
Priorität im Projekt: Ziel der Zwischenabgabe war die funktionale Umsetzung und die Erfüllung der Anforderungen laut Spezifikation/Checkliste – nicht die pixelgenaue Nachbildung des Mockups. Dieses Ziel wurde erreicht. Abweichungen zum Figma-Layout sind daher beabsichtigt möglich und keine Qualitätsmängel per se. Für die Finalabgabe gilt dieselbe Linie: Spezifikation und fachliche Vollständigkeit stehen über 1:1-Optik zum Mockup; das Mockup bleibt die Orientierung für UX und Branding.

5.1 Vergleich Mockup und tatsächliche Implementierung

Abbildung 1 – Dashboard „Meine Touren“

Zweck: Übersicht aller Touren, Filter nach Transporttyp, Suche, CTA „Neue Tour“, ausgewählte Tour mit Detail-Kennzahlen und Log-Liste; Karten-Platzhalter.

Mockup:



Implementierung:

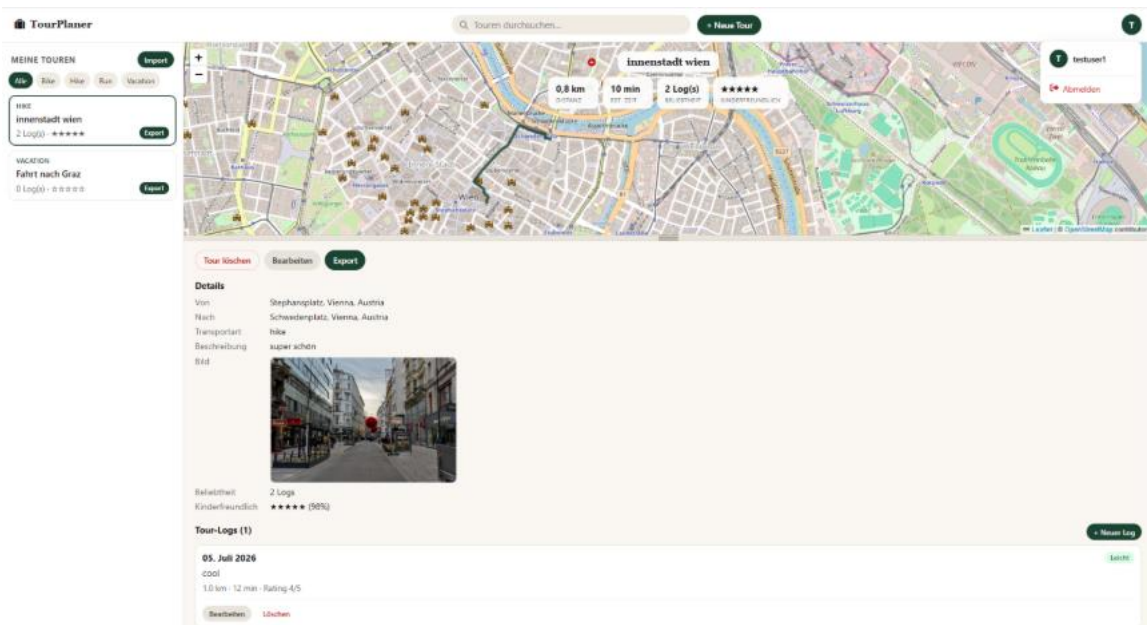
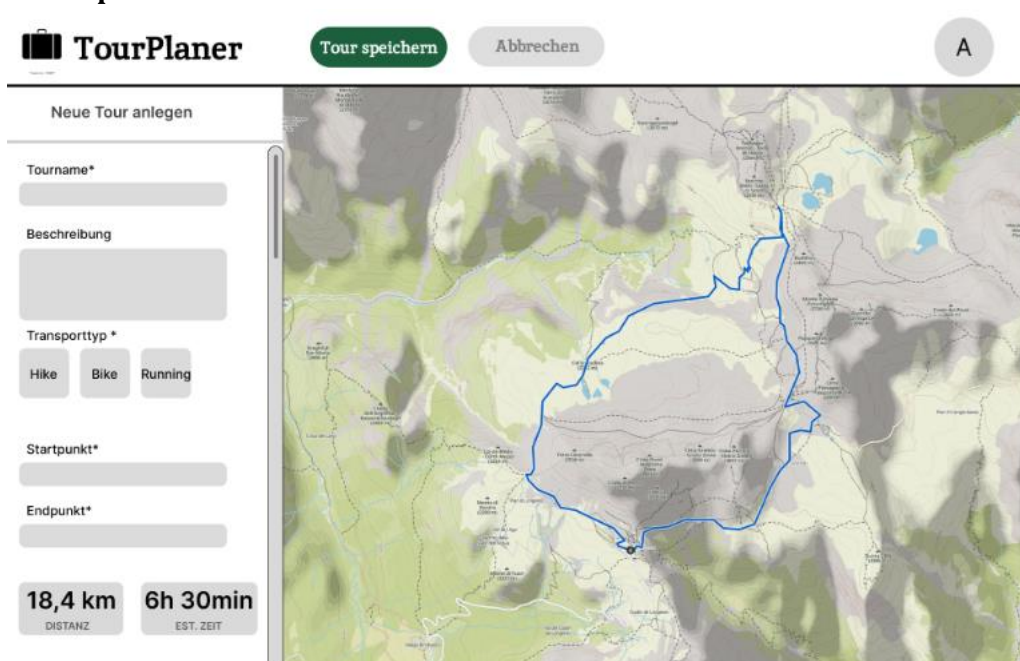


Abbildung 2 – Tour anlegen / bearbeiten

Zweck: Erfassung Name, Beschreibung, Transport, Start/Ziel, Distanz/Zeit, Bild-URL, Routen-Info; Speichern/Abbrechen; Karten-Platzhalter.

Mockup:



Implementierung:

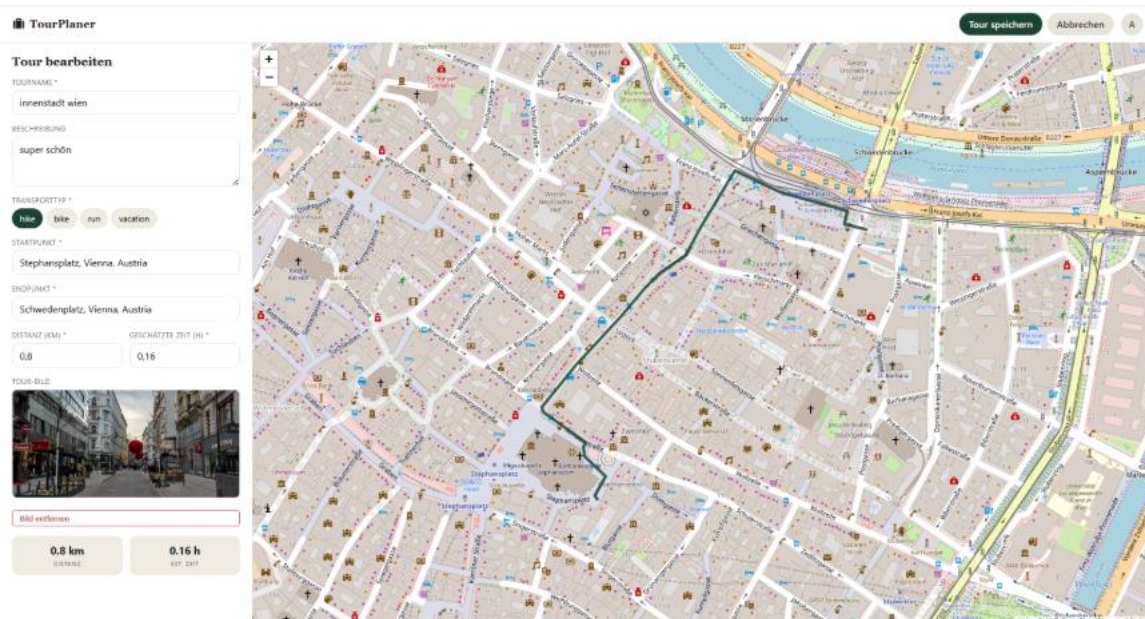
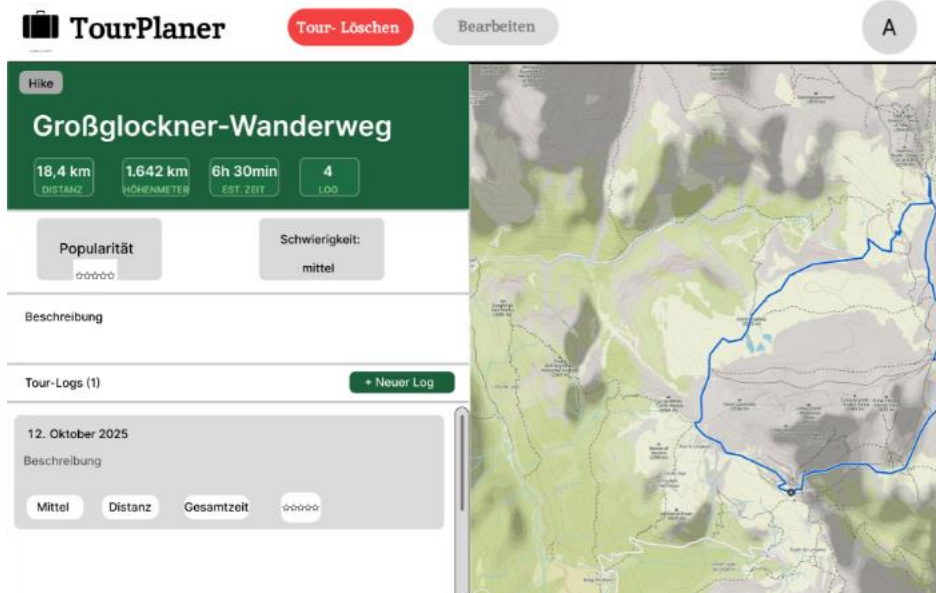


Abbildung 3 – Tour-Detail mit Logs

Zweck: Fokus auf Beschreibung, Metriken, Log-Liste mit Aktionen.

Mockup:



Implementierung: Siehe Abbildung 1

Abbildung 4 – Log anlegen / bearbeiten

Zweck: Datum/Zeit, Kommentar, Schwierigkeit, Distanz, Zeit, Bewertung; Kontext der Tour sichtbar.

Mockup:

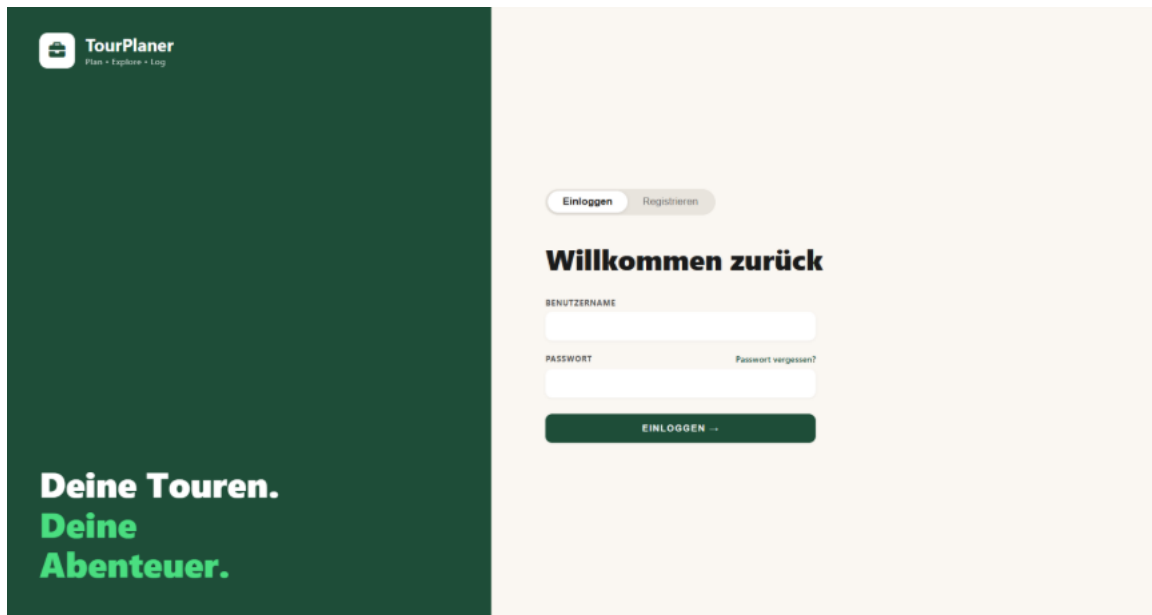
The mockup shows the 'TourPlanner' app interface. At the top, there's a header with the app name and two buttons: 'Log speichern' and 'Abbrechen'. Below the header, a green bar indicates the activity type 'Hike' and the title 'Großglockner-Wanderweg'. To the right of the title, four statistics are displayed: '18,4 km DISTANZ', '1.642 km HÖHENMETER', '6h 30min EST. ZEIT', and '4 LOG'. The main form on the left includes fields for 'Datum*' (12. Februar 2025), 'Startzeit' (8:30), 'Kommentar*' (empty text area), 'Schwierigkeit*' (three buttons: 'Leicht', 'Mittel', 'Schwer'), 'Zurückgelegte Distanz*' (17,65 km), 'Gesamtzeit*' (6h 30min), and 'Bewertung*' (five empty star icons). On the right, a map shows a blue trail route through a mountainous area.

Implementierung:

The implementation shows the actual app interface. The header includes the 'TourPlanner' logo and buttons for 'Log speichern', 'Abbrechen', and a user icon 'A'. The main form is titled 'Neuer Log - innenstadt wien'. It includes fields for 'Datum & Uhrzeit*' (06.07.2026 12:55), 'Kommentar*' (Es war ein schöner Spaziergang.), 'Schwierigkeit*' (three buttons: 'Leicht*', 'Mittel', 'Schwer'), 'Distanz (km)*' (0,8), 'Gesamtzeit (h)*' (0,2), and 'Bewertung*' (5 stars). On the right, a map shows a green trail route through a city street grid.

Abbildung 5/6 – Login / Registrierung

Mockup und Implementierung ident:



The login form is part of a web application for 'TourPlaner'. The left sidebar is dark green with the 'TourPlaner' logo (a white suitcase icon) and the text 'Plan • Explore • Log'. Below the logo, the slogan 'Deine Touren. Deine Abenteuer.' is displayed in white and green. The main content area is light beige. At the top, there are two buttons: 'Einloggen' (active) and 'Registrieren'. The heading 'Willkommen zurück' is in bold black. Below it, the 'BENUTZERNAME' field is a white input box. The 'PASSWORT' field is a white input box with a 'Passwort vergessen?' link to its right. At the bottom is a dark green button with the text 'EINLOGGEN →'.

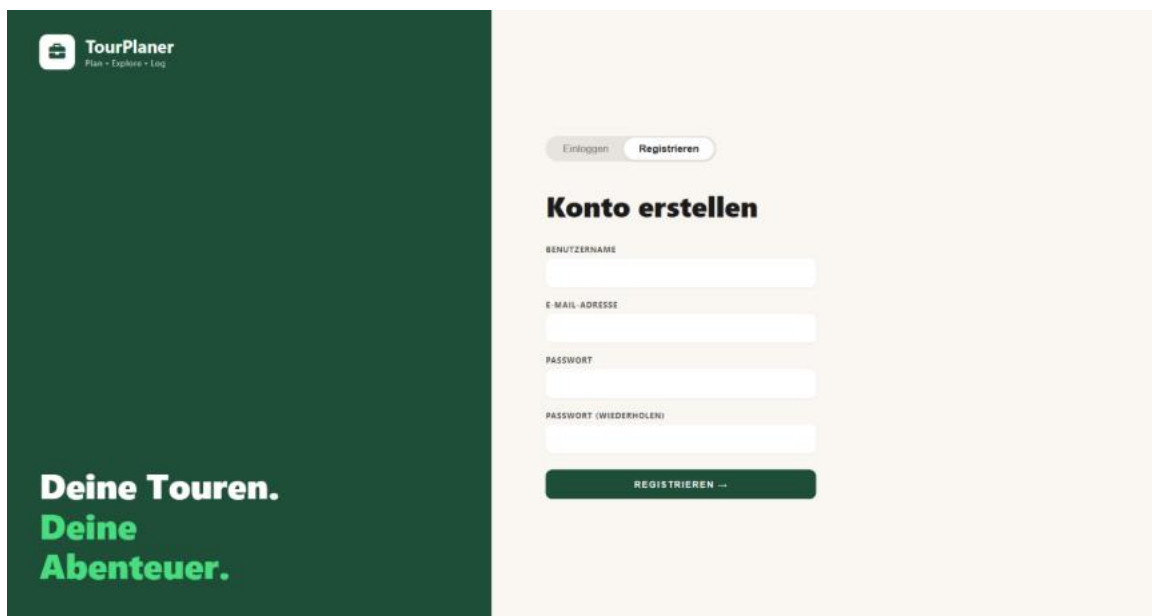
TourPlaner
Plan • Explore • Log

Willkommen zurück

BENUTZERNAME

PASSWORT [Passwort vergessen?](#)

EINLOGGEN →



The registration form is part of the same 'TourPlaner' web application. The left sidebar is identical to the login form. The main content area is light beige. At the top, there are two buttons: 'Einloggen' and 'Registrieren' (active). The heading 'Konto erstellen' is in bold black. Below it, the 'BENUTZERNAME' field is a white input box. The 'E-MAIL-ADRESSE' field is a white input box. The 'PASSWORT' field is a white input box. The 'PASSWORT (WIEDERHOLEN)' field is a white input box. At the bottom is a dark green button with the text 'REGISTRIEREN →'.

TourPlaner
Plan • Explore • Log

Konto erstellen

BENUTZERNAME

E-MAIL-ADRESSE

PASSWORT

PASSWORT (WIEDERHOLEN)

REGISTRIEREN →

6. Bibliotheksentscheidungen

Backend:

Bibliothek	Warum gewählt	Lessons Learned
Spring Boot 4	Standard für REST + DI + Security im Java-Ökosystem	Konfiguration über application-local.properties für Secrets (JWT, API-Key)
Spring Data JPA	Schnelle PostgreSQL-Anbindung, wenig Boilerplate	ddl-auto=update nur für Entwicklung; Produktion braucht Migrations
Spring Security + JWT	Stateless JWT-Auth für SPA	userId im Token-Claim für Multi-Tenancy
Lombok	Weniger Getter/Setter in Entities/DTOs	Build muss Annotation Processor korrekt konfigurieren
PostgreSQL	Relationale Daten, Docker-Setup vorhanden	Docker Compose vor Backend-Start starten
Jackson	JSON-Serialisierung für REST und Export/Import	GeoJSON als String in DB speichern

Frontend:

Bibliothek	Warum gewählt	Lessons Learned
Angular 21	Vorgabe/Kurs, Standalone Components, Signals	Lazy Loading mit loadChildren hält Bundle klein
Leaflet	Leichtgewichtige Open-Source-Karten	GeoJSON-Layer für Routen; OSM-Tiles kostenlos
RxJS	Debounced Search (350 ms), HTTP-Pipelines	switchMap verhindert Race Conditions bei Suche
Vitest	Angular 21 Standard-Testrunner	Frontend-Tests noch minimal – mehr Coverage wäre sinnvoll

Extern:

Dienst	Warum	Lessons Learned
OpenRouteService	Kostenloses Geocoding + Routing	API-Key nötig; graceful degradation wenn Key fehlt
Font Awesome (CDN)	Icons ohne npm-Paket	CDN-Abhängigkeit bei Offline-Dev

7. Implementierte Design Patterns

Pattern	Verwendung im Projekt
Layered Architecture (n-Tier)	Controller → Service → DAL → Repository → DB
Repository Pattern	TourRepository, TourLogRepository, UserRepository
DTO Pattern	Trennung von API (TourDto) und Domäne (Tour)
Domain Model	Tour/TourLog getrennt von JPA-Entities
Dependency Injection	Constructor Injection in allen Spring-Komponenten
Filter Chain	JwtAuthFilter vor Spring Security
Facade	OpenRouteTourRouteEnrichment kapselt externe HTTP-Calls
Exception Translation	DAL-Exceptions → Service-Exceptions → GlobalExceptionHandler → HTTP-Status
Guard (Frontend)	authGuard schützt Routen vor unauthentifiziertem Zugriff
Interceptor	authInterceptor hängt JWT an jeden HTTP-Request

8. Unit-Testing-Entscheidungen

Strategie:

- Fokus auf Backend-Service-Schicht – dort liegt die meiste Business-Logik
- Mockito + JUnit 5 – DAL und externe Integration werden gemockt
- AssertJ – lesbare Assertions
- H2 – nur als Test-Dependency verfügbar; aktuelle Tests nutzen primär Mocks

Vorhandene Tests:

Testklasse	Was wird getestet
JpaTourServiceTest	Volltextsuche, Popularität, Child-Friendliness-Berechnung
JpaTourLogServiceTest	Log-CRUD, Ownership-Checks
JpaAuthServiceTest	Register (Duplikat), Login (falsche Credentials)
OpenRouteTourRouteEnrichmentTest	ORS-Integration (mit Mocks)
TourplannerApplicationTests	Spring Context lädt
app.spec.ts (Frontend)	Smoke-Test: App-Komponente erstellt

Bewusste Entscheidungen:

- Keine Controller-Integrationstests – Fokus auf testbare Service-Logik
- Frontend minimal getestet – UI-Logik überwiegend manuell; Vitest-Setup vorhanden
- Berechnete Metriken getestet – Child-Friendliness und Suche sind Kernfeatures

9. Unique Feature

Adress-Autovervollständigung mit Live-Routenvorschau

Die Angabe fordert nur, dass Distanz und Zeit per OpenRouteService berechnet und die Route mit Leaflet dargestellt werden. Wir gehen darüber hinaus: Bereits beim Ausfüllen des Tour-Formulars (TourFormComponent) werden Adresseingaben debounced (350ms) an einen neuen Backend-Endpunkt /api/geocode/search (GeocodingController, OpenRouteTourRouteEnrichment.searchLocations) geschickt, der Adressvorschläge von OpenRouteService zurückliefert. Wählt der Nutzer einen Vorschlag, sind dessen Koordinaten sofort bekannt, wodurch ein zweiter Endpunkt /api/route-preview die Route mit nur einem einzigen Directions-Call berechnen kann (previewRouteByCoordinates), statt wie sonst nötig zwei zusätzliche Geocoding-Calls durchzuführen. Das Ergebnis (Distanz, Zeit, Routen-GeoJSON) wird direkt ins Formular übernommen und die eingebettete Leaflet-Karte (TourRouteMapComponent) aktualisiert sich über Angulars ngOnChanges live – der Nutzer sieht die Route also schon während der Eingabe, bevor die Tour überhaupt gespeichert wird. Beim tatsächlichen Speichern (JpaTourService/OpenRouteTourRouteEnrichment.enrichIfPossible) wird der bereits vorhandene routeInfo-Wert wiederverwendet, um einen redundanten API-Call zu vermeiden. Technisch kombiniert dieses Feature RxJS (debounceTime, switchMap zum Abbrechen veralteter Requests), parallele HTTP-Calls im Backend (CompletableFuture) sowie reaktive Zwei-Wege-Datenbindung zwischen Formular und Kartenkomponente.

10. Zeitaufwand (Schätzung / Ist)

Arbeitspaket	Stunden (Ist/Schätzung)
Backend REST + DTO	10/12
Angular Dashboard + Routing	18/22
Formulare + Validierung	12/15
Integration, CORS, Fehlersuche, UI-Feinheiten	6/8
Wireframes / Protokoll / README	8/9
Backend: PostgreSQL/JPA-DAL-Migration + eigene Exceptions je Layer	7/6
Authentifizierung (Backend: Spring Security/JWT, Frontend: Login/Register-UI, Guard, Interceptor)	9/7
OpenRouteService + Leaflet-Karte + Adresssuche/Live-Routenvorschau	10/8
Tourattribute (Popularity, Child-Friendliness) + Volltextsuche inkl. dieser Werte	6/5
Bild-Upload (Dateisystem) + Import/Export (JSON)	5/4
Logging-Framework (Logback, Rolling File, Profile)	2/2
Unit-Tests (25 JUnit-Tests, Service-Layer)	5/4
Protokoll final (UML: Use-Case/Klassen/Sequenzdiagramm), Wireframes-Update, Bugfixing/Feinschliff	4/4
Summe	102/106

11. Git-Historie

Die Entwicklungsgeschichte ist über das Git-Repository nachvollziehbar (Commits, Branches, Merges). Eine Kopie der Historie ins PDF ist laut Aufgabenstellung nicht nötig. Verweis auf das Repo genügt.